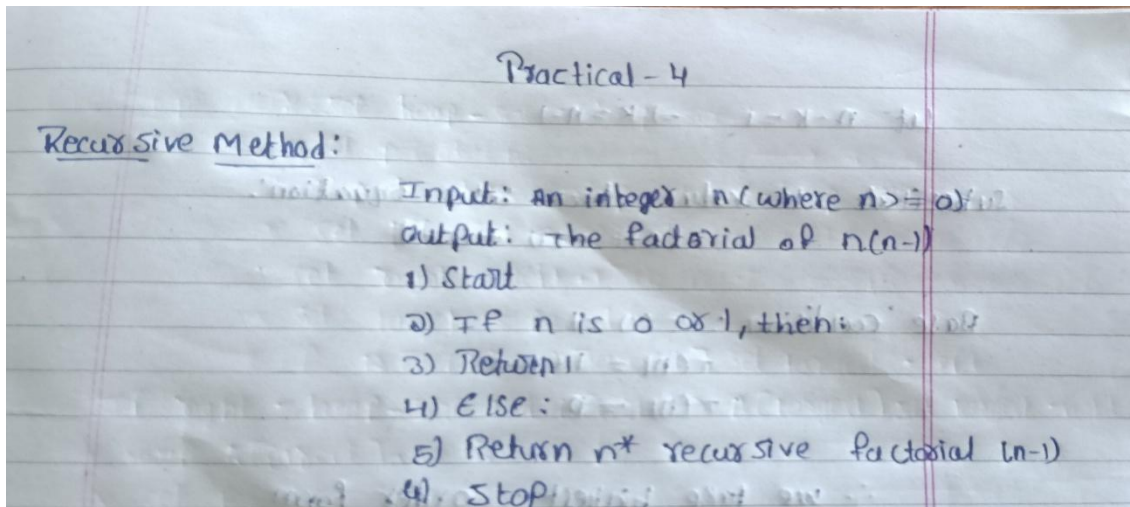


Practical - 4

Aim : To implement and analyze the execution time of the factorial algorithm using both iterative and recursive approaches **1)**

Recursive Method :



Program / Code :

```
#include <iostream> using
namespace std; class
Factorial
{
public:
    int factr(int n)
    {
        if (n == 0 || n == 1)
        {
            return 1;
        }
    }
}
```

```
else{
    int result = n * factr(n - 1);
return result;
}
};
int main()
{
    int n, result;
    // Create an object of the class
    Factorial obj;
    cout << "Enter the value to find its factorial: ";
    cin >> n;    result = obj.factr(n);
    cout << "Factorial of " << n << " is " << result << endl;
    return 0;
}
```

Output :

```
Enter the value to find its factorial: 23
Factorial of 23 is 862453760
```

```
=== Code Execution Successful ===
```

Time Complexity :

CH.Sivamani 92400118547 , 5EN_11_LAB-A

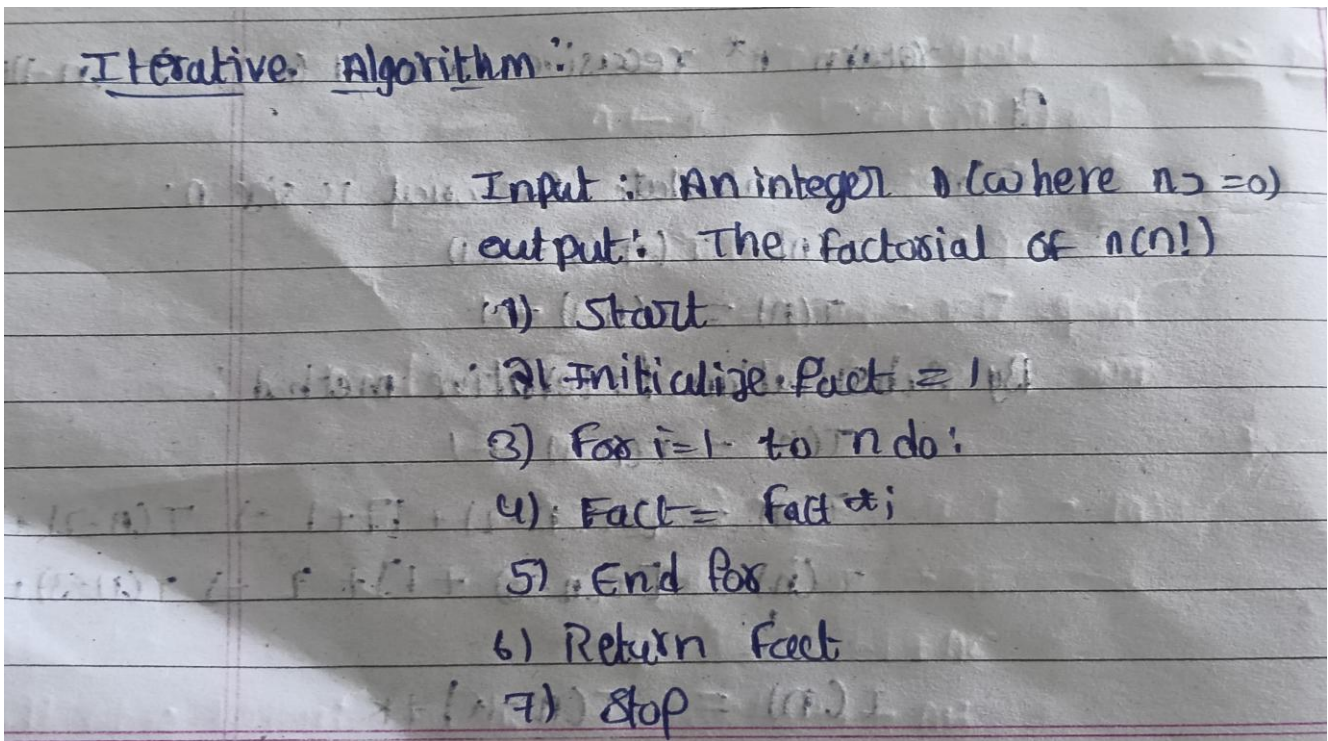
Best Case : $O(n)$

Average Case : $O(n)$

Worst Case : $O(n)$

2) Iterative Method :

Algorithm :



Iterative Algorithm:

Input : An integer n (where $n \geq 0$)
(output:) The factorial of $n(n!)$

- 1) Start
- 2) Initialize $Fact = 1$
- 3) For $i = 1$ to n do:
- 4) $Fact = Fact * i$
- 5) End For
- 6) Return $Fact$
- 7) Stop

Program / Code :

```
#include <iostream> using  
namespace std; class  
Factorial
```

{CH.Sivamani 92400118547 , 5EN_11_LAB-A

public:

```
int fact(int n)
{
    int result = 1;    for
(int i = 1; i <= n; i++)
    {
        result = result * i;
    }

return result;
}
};
int main()
{
    int n,result;
    Factorial obj;
    cout << "Enter the value to find its factorial: ";
    cin >> n;    result = obj.fact(n);
    cout << "Factorial of " << n << " is " << result << endl;
return 0;
}
```

Output :

```
Enter the value to find its factorial: 12
Factorial of 12 is 479001600
```

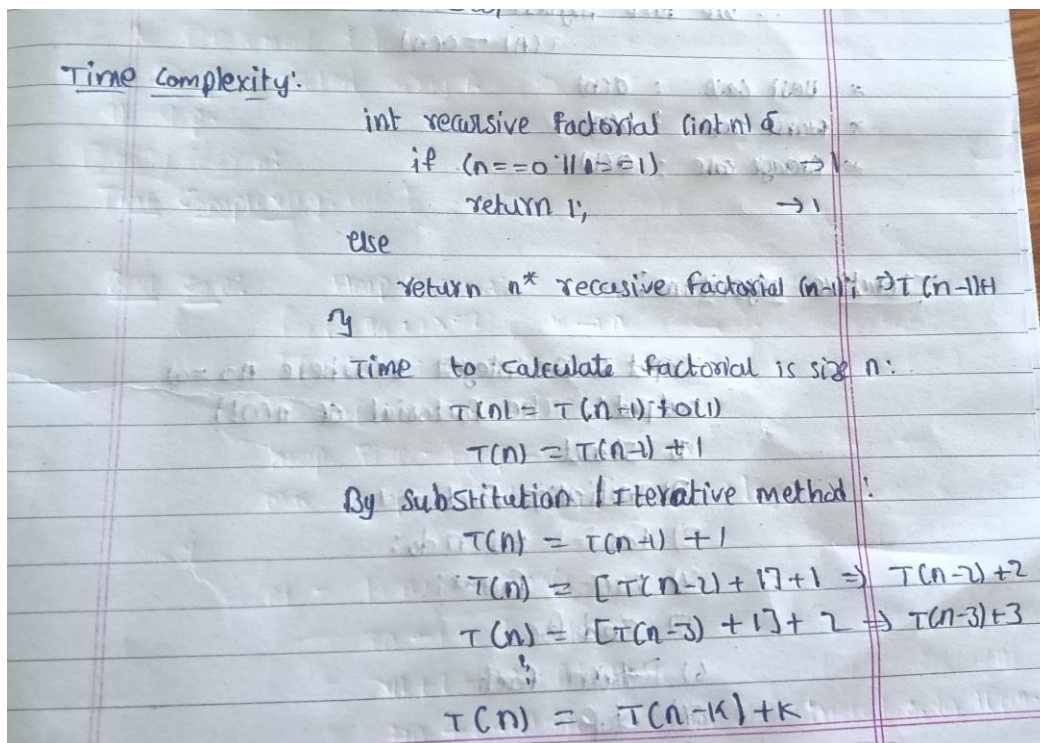
```
=== Code Execution Successful ===
```

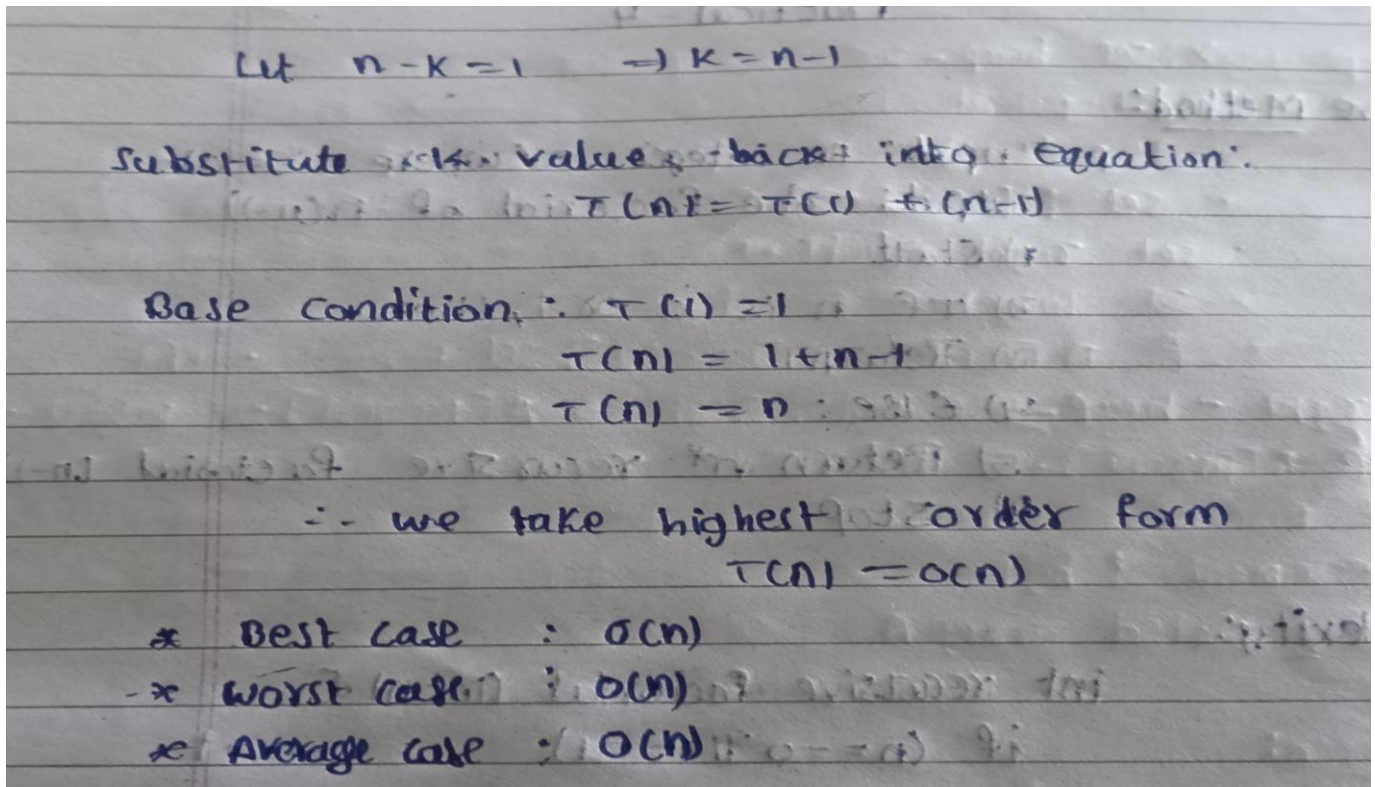
Time complexity :

Best Case : $O(n)$

Average Case : $O(n)$

Worst Case : $O(n)$





Conclusion : The iterative method is faster and more reliable because it uses a direct loop. The recursive method is slower because managing multiple function calls takes more time.